prefix and KMP's example

UVA12467 Secret Word

题目描述

给定一个字符串 S，要求出一个字符串 C 满足以下条件：

- C 是 S 的非空子串（可以等于 S）；
- S 的开头为反转的 C；
- C 为满足上述条件的最长子串。

你的思路在逻辑上是**完全正确**的，确实可以通过遍历长度并逐一检查来找到答案。

不过，这种方法的时间复杂度是 $O(N^2)$（遍历 N 次，每次做一次字符串匹配或反转，通常是 $O(N)$），如果字符串 S 的长度达到 10^5 级别，这种做法会超时（TLE）。

我们可以利用 **KMP 算法的预处理逻辑**，在 $O(N)$ 的时间复杂度内解决这个问题。

优化思路

1. **转化问题：**
   - 题目要求：S 开头为 C^R（C 的反转）。这意味着 C^R 是 S 的一个**前缀**。
   - 令 $P = S[0 \dots |C| - 1]$ 为 S 的一个前缀。
   - 那么 $C^R = P$，即 $C = P^R$。
   - 题目要求 C 是 S 的子串，即 **P^R 是 S 的子串**。
   - 因此，问题转化为：**找出最长的前缀 P，使得 P^R 是 S 的子串。**
   - 进一步推导：P^R 是 S 的子串，等价于 **P 是 S^R（S 的反转字符串）的子串**。

   所以，题目最终变成了：**在 S^R 中，寻找 S 的最长前缀。**
2. **使用 KMP 算法：**
   - 我们可以把 S 看作 KMP 算法中的**模式串（Pattern）**。
   - 把 S^R 看作 KMP 算法中的**文本串（Text）**。
   - 当我们在 S^R 中匹配 S 时，KMP 的匹配状态（`j` 指针的值）实际上就代表了“当前匹配到的 S 的前缀长度”。

- 我们只需要在匹配过程中，记录 `j` 指针出现过的**最大值**，这个最大值就是我们要求的 C 的最长长度。

具体步骤

1. 将原字符串 S 反转，得到 S^R。
2. 对字符串 S（作为模式串）构建 KMP 的 `next` 数组（或 `pi` 数组）。
3. 利用构建好的 `next` 数组，在 S^R（作为文本串）中进行匹配。
4. 在匹配过程中，每当匹配成功一位（`j` 增加），就记录当前 `j` 的值。
5. 匹配结束后，记录下来的 `j` 的最大值，就是 C 的最大长度。

为什么这样做更快？

- **你的原方法：** 每一轮都在做字符串匹配，就像是一个嵌套循环，导致 $O(N^2)$。
- **KMP 方法：** 只需要预处理一次 S，然后遍历一次 S^R，整个过程只需要 $O(N)$ 的时间。

举例说明

假设 $S = \text{"abacaba"}$

1. $S^R = \text{"abacaba"}$
2. 模式串 $S = \text{"abacaba"}$，文本串 $S^R = \text{"abacaba"}$。
3. KMP 匹配过程中，由于两者相同，最大匹配长度可以达到 7。
4. 答案就是 7。

再假设 $S = \text{"abc"}$

1. $S^R = \text{"cba"}$
2. 模式串 $S = \text{"abc"}$，文本串 $S^R = \text{"cba"}$。
3. 匹配过程：
   - $S[0]$ ('a') 在 S^R 中能找到（在索引 2 处），`j` 最大值变为 1。
   - $S[0..1]$ ('ab') 在 S^R 中找不到。
4. 答案就是 1。

这种方法逻辑简洁，且在算法竞赛中是处理此类问题的标准高效解法。

UVA11019 Matrix Matcher

题目描述

T 组数据，每组数据给定一个 $N \times M$ 的 A 矩阵，满足 $1 \le N, M \le 1000$，再给定一个 $X \times Y$ 的 B 矩阵，满足 $1 \le X, Y \le 100$，求 B 矩阵在 A 矩阵中出现的次数。

你提出的思路是“朴素匹配”的变体。虽然直觉上很合理，但我们需要仔细分析一下**复杂度**。

如果按照你的方案：

1. 用 KMP 在 A 的每一行里找 B 的第一行：这部分是 $O(N \cdot M)$。
2. 只要匹配成功，就进入循环检查 B 的剩下 $X-1$ 行。

问题在于：
如果数据构造得很“极端”（例如 A 全是 0，B 全是 0），你的代码会在 A 的每一行都频繁进入“检查剩下 $X-1$ 行”的过程。
这种情况下，最坏复杂度会退化到 $O(N \cdot M \cdot X \cdot Y)$。
对于 $N, M = 1000$，$X, Y = 100$ 的数据规模，总操作数可能会达到 10^{10} 量级，这在一般的算法竞赛中是**绝对会超时（TLE）**的。

那么，标准的 2D 矩阵匹配算法应该怎么做？

针对这种二维矩阵匹配，通常有两种主流的“标准解法”，比简单的“一行行对比”要快得多：

方案一：二维滚动哈希（Rolling Hash）—— 最推荐，最简单

这是解决二维矩阵匹配最常用的方法，代码量小且效率高。

- **思路：**
  i. 将 B 矩阵看作一个巨大的“数字”，计算它的 Hash 值。
  ii. 对于 A 矩阵中每一个大小为 $X \times Y$ 的子矩阵，我们也计算它的 Hash 值。
  iii. 通过二维滚动哈希，可以在 $O(N \cdot M)$ 的时间内算出 A 中所有 $X \times Y$ 子矩阵的 Hash 值。
  iv. 只要 Hash 值相等，就认为是匹配成功。
- **复杂度：** $O(N \cdot M)$。
- **优点：** 几乎不需要复杂的逻辑判断，写起来非常快。

方案二：KMP 的二维扩展（哈希 + KMP）

如果你坚持想用 KMP 思想，可以把二维问题降维成一维问题：

- **思路：**
  i. **预处理行：** 把 B 矩阵的每一行看作一个“字符串”。因为行数 $X \leq 100$，你可以先对 B 的 X 行做 Hash，或者直接建立 Aho-Corasick 自动机（如果行种类很多）。
  ii. **降维匹配：** 对 A 矩阵的每一行进行预处理。对于 A 的第 i 行，找到所有和 B 的某一行完全匹配的位置，并记录下来。
  iii. **变成一维 KMP：** 这样 A 矩阵就变成了一个“代号矩阵”，其中每个位置存的是“这一行匹配到了 B 的哪一行”。
  iv. 最后，对这个“代号矩阵”的每一列，用 KMP 算法去匹配 B 的那 X 行的代号序列。
- **复杂度：** 同样是 $O(N \cdot M)$。
- **优点：** 纯算法逻辑，不需要处理哈希碰撞的概率问题。

总结建议

1. **千万不要直接用“匹配到第一行再去暴力检查剩下行”的方法**，那个在特殊构造的数据（如全 0 矩阵）下是必死无疑的。
2. 如果你是在做题（如洛谷等OJ），**二维滚动哈希**是性价比最高的方案，代码很短，不容易写错。
3. 如果面试官问你，你可以回答：“用暴力检查效率太低，我会通过**二维滚动哈希**在 $O(N \cdot M)$ 时间内预处理出所有子矩阵的哈希值进行比对；或者通过**行哈希+KMP**将二维降维为一维来处理。”

如果你需要二维滚动哈希的代码模板或者更详细的解释，我可以给你展示一下！